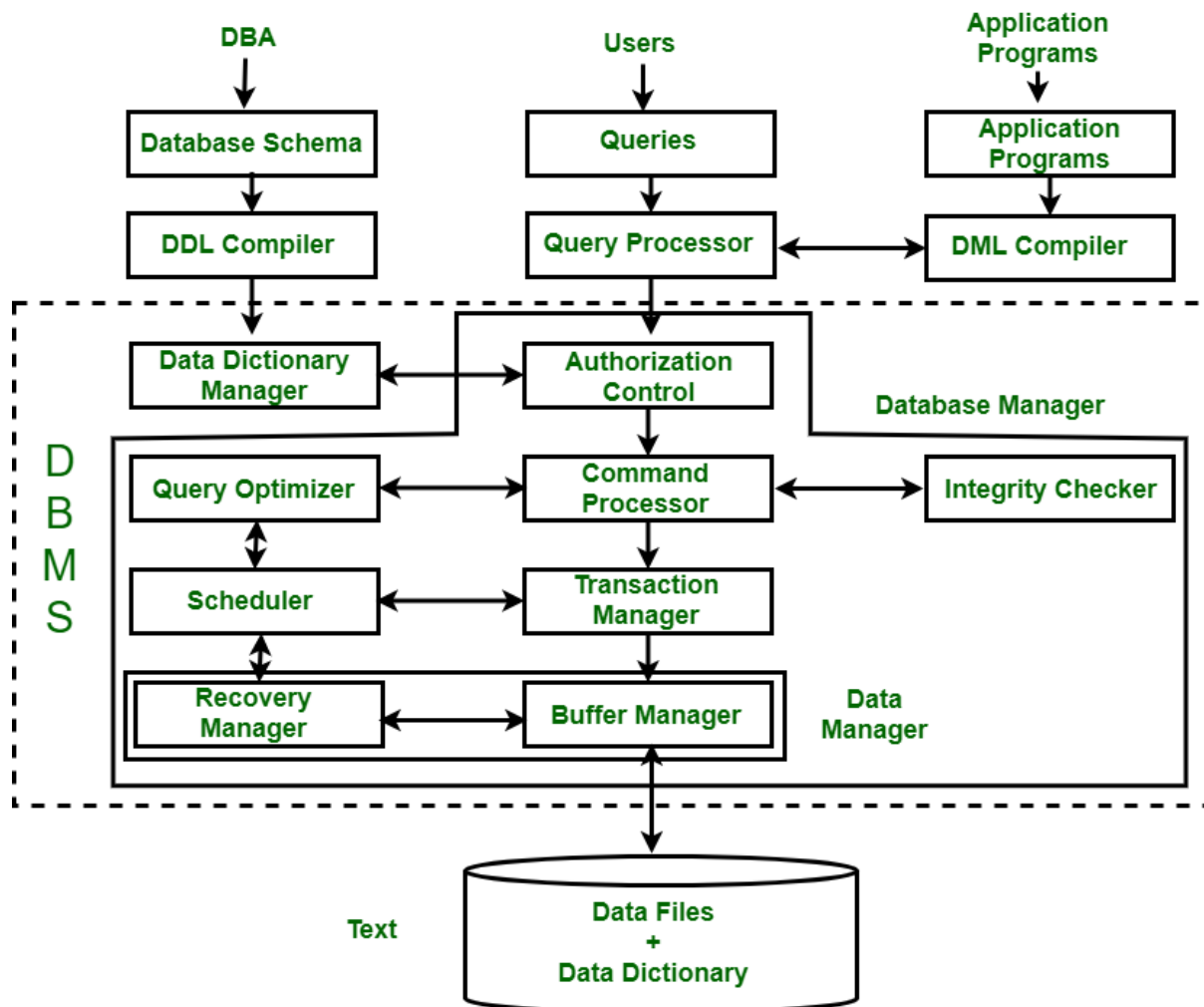


Database Architecture/ Overall System Structure

- **Database Architecture** refers to the internal components of the DBMS, including the Query Processor, Storage Manager, and Disk Storage.
- It also defines the interaction of these components.

Components of a Database System



Text

Architecture of DBMS

1. Query Processor

- It **interprets the requests (queries) received from end user** via an application program into instructions.
- It also executes the user request which is received from the **DML compiler**.
- Query Processor contains the following components -
 - **DML Compiler:** It processes the DML statements into low level instruction (machine language), so that they can be executed.
 - **DDL Interpreter:** It processes the DDL statements into a set of table containing meta data (data about data).
 - **Embedded DML Pre-compiler:** It processes DML statements embedded in an application program into procedural calls.
 - **Query Optimizer:** The Query Optimizer executes instructions generated by the DML Compiler and improves query execution efficiency by choosing the best query plan, considering factors such as indexing, join order, and available system resources. For instance, if a query involves joining two large tables, the optimizer will select the **best join order to minimize query execution time**.
-

2. Storage Manager

- Storage Manager is an **interface between the data stored in the database and the queries received**.
- It is also known as **Database Control System**.
- It maintains the consistency and integrity of the database by applying the constraints and executing the [DCL](#) statements.
- It is responsible for updating, storing, deleting, and retrieving data in the database.
- It contains the following components:
 - **Authorization Manager:** It ensures role-based access control, i.e., checks whether the particular person is privileged to perform the requested operation or not.

- **Integrity Manager:** It checks the **integrity constraints** when the database is modified.
- **Transaction Manager:** It controls concurrent access by performing the operations in a scheduled way when it receives the transaction. Thus, it ensures that the database remains in the consistent state before and after the execution of a transaction.
- **File Manager:** It manages the file space and the data structure used to represent information in the database.
- **Buffer Manager:** It is responsible for **cache memory** and the transfer of data between the secondary storage and main memory.

3. Disk Storage

It contains the following essential components:

- **Data Files:** It stores the actual data in the database.
- **Data Dictionary:** It contains the information about the **structure of database objects** such as tables, constraints, and relationships. It is the repository of information that governs the metadata.
- **Indices:** Provides **faster data retrieval** by allowing the DBMS to find records quickly, improving query performance.

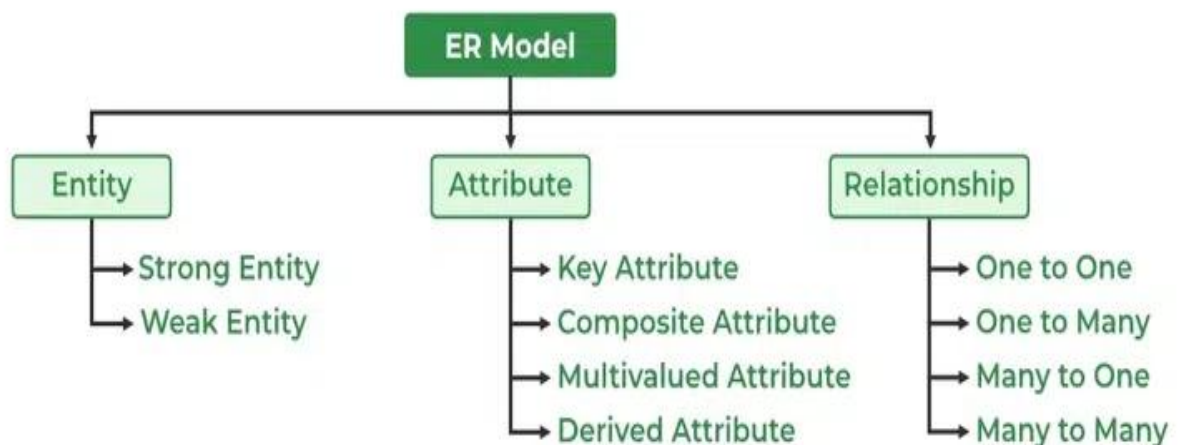
Entity-Relationship (E-R) Model

Introduction

- The Entity-Relationship Model (ER Model) is a **conceptual model** for designing a database.
- This model **represents the logical structure** of a database, including entities, their attributes, and relationships between them.
- **Entity:** An object that is stored as data. E.g: *Student*, *Course*, or *Company*.
- **Attribute:** Properties that describe an entity.

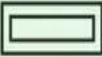
E.g: *StudentID*, *CourseName*, or *EmployeeEmail*.

- **Relationship:** A connection between entities. E.g: *Student* enrolls in a *Course*.



Symbols Used in ER Model

ER Model is used to model the logical view of the system from a data perspective, which consists of these symbols:

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Symbols used in ER Diagram

• Entity

- An Entity represents a **real-world object**, concept or thing about which data is stored in a database.
- It acts as a **building block of a database**.
- **Tables in a relational database represent these entities.**

Example of entities:

- **Real-World Objects:** Person, Car, Employee etc.

The entity type defines the structure of an entity, while individual instances of that type represent specific entities.

• Entity Set

An entity refers to an **individual object of an entity type**, and the collection of all entities of a particular type is called an entity set.

For example, E1 is an entity that belongs to the entity type "Student," and the group of all students forms the entity set.

Types of Entity

There are two main types of entities:

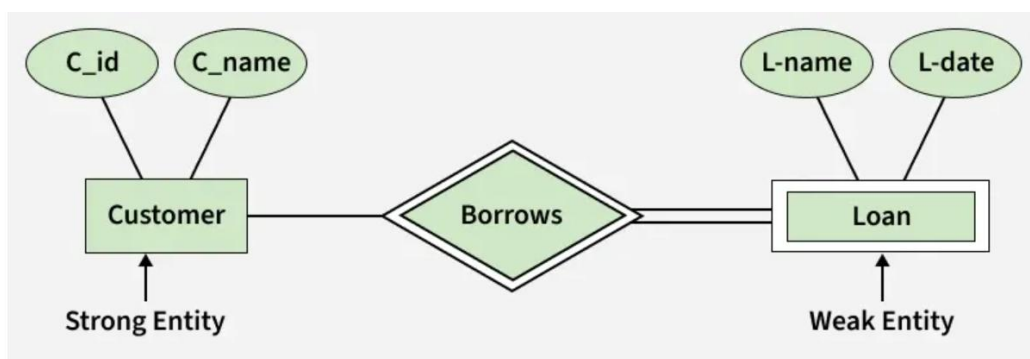
1. Strong Entity

- A Strong Entity is a type of entity that has a **key Attribute that can uniquely identify each instance of the entity.**
- A Strong Entity does not depend on any other Entity in the Schema for its identification.
- It has a **primary key that ensures its uniqueness** and is represented by a rectangle in an ER diagram.

2. Weak Entity

- A **Weak Entity cannot be uniquely identified** by its own attributes alone.
- It **depends on a strong entity** to be identified.
- A weak entity is associated with an identifying entity (strong entity), which helps in its identification.
- A weak entity are represented by a **double rectangle**.
- The participation of weak entity types is always total.
- The relationship between the weak entity type and its identifying strong entity type is called an identifying relationship, and it is represented by a double diamond.

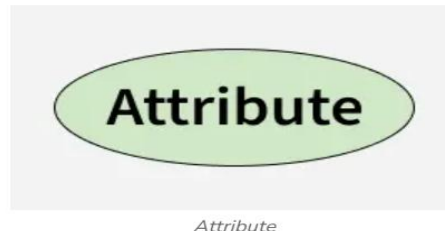
Example:



Strong Entity and Weak Entity

Attributes in ER Model

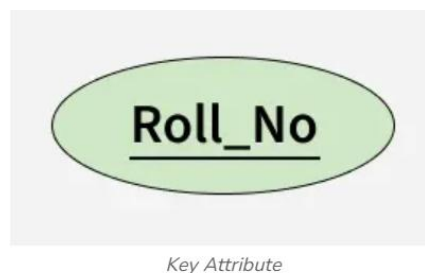
- Attributes are the properties that define the entity type.
- For example, for a Student entity Roll_No, Name, DOB, Age, Address, and Mobile_No are the attributes that define entity type Student.
- In ER diagram, the attribute is represented by an oval.



Types of Attributes

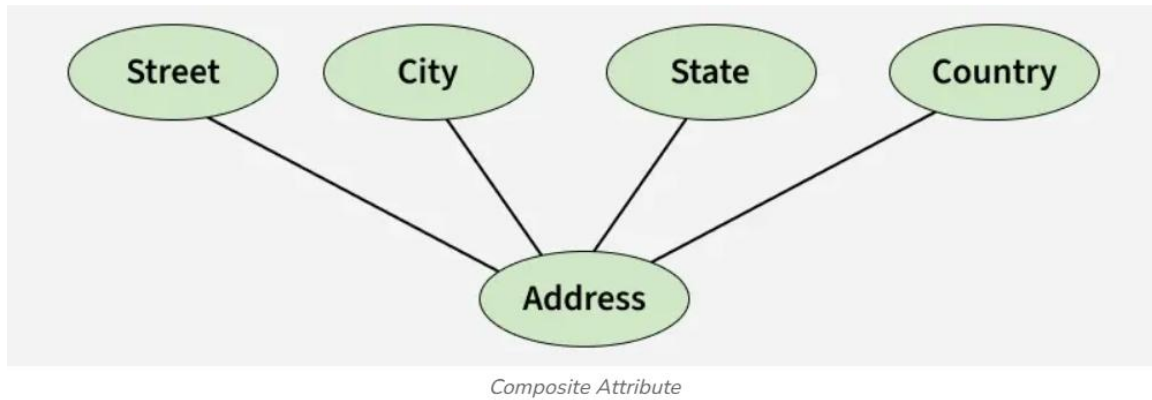
1. Key Attribute

- The attribute which uniquely identifies each entity in the entity set is called the key attribute.
- For example, Roll_No will be unique for each student.
- In ER diagram, the key attribute is represented by an oval with an underline.



2. Composite Attribute

- An attribute composed of many other attributes is called a composite attribute.
- For example, the Address attribute of the student Entity type consists of Street, City, State, and Country.
- In ER diagram, the composite attribute is represented by an oval comprising of ovals.

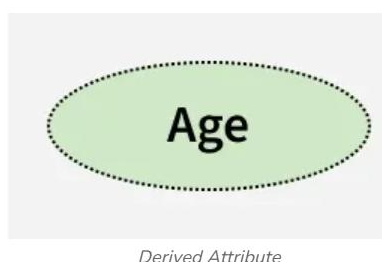


3. Multivalued Attribute

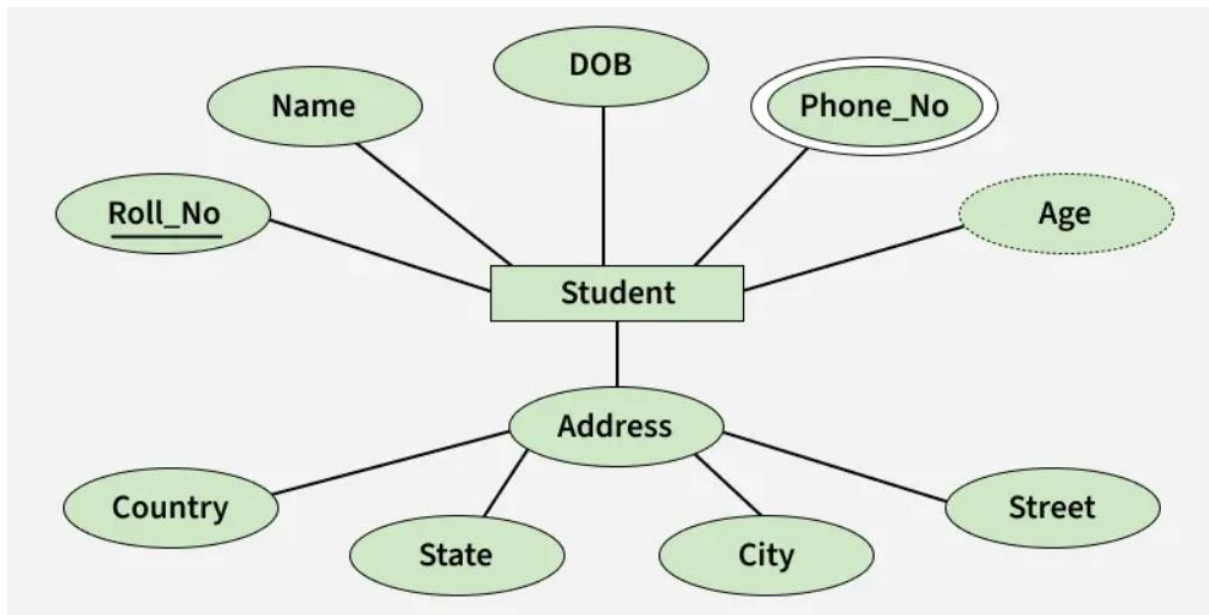
- An attribute consisting of more than one value for a given entity.
- For example, Phone_No (can be more than one for a given student).
- In ER diagram, a multivalued attribute is represented by a double oval.



- An attribute that can be derived from other attributes of the entity type is known as a derived attribute. e.g.; Age (can be derived from DOB).
- In ER diagram, the derived attribute is represented by a dashed oval.



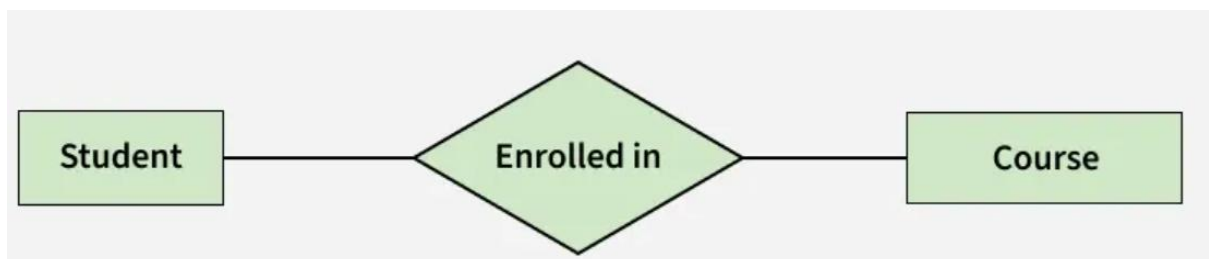
The Complete Entity Type Student with its Attributes can be represented as:



Entity and Attributes

Relationship Type and Relationship Set

- A Relationship Type represents the association between entity types.
- For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course.
- In ER diagram, the relationship type is represented by a diamond and connecting the entities with lines.

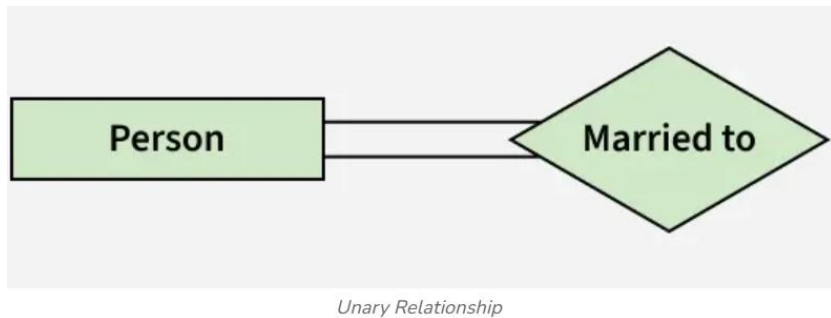


Degree of a Relationship Set

The **number of different entity sets participating in a relationship set** is called the degree of a relationship set.

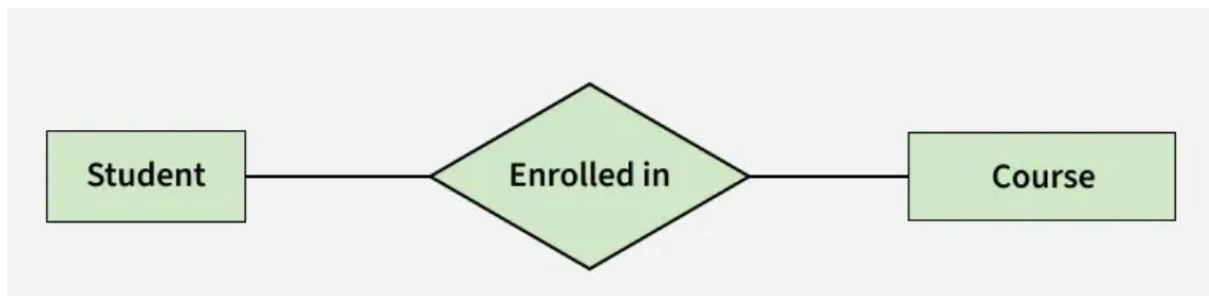
1. Unary/Recursive Relationship:

- When there is only ONE entity set participating in a relation, the relationship is called a unary relationship.
- For example, one person is married to only one person.



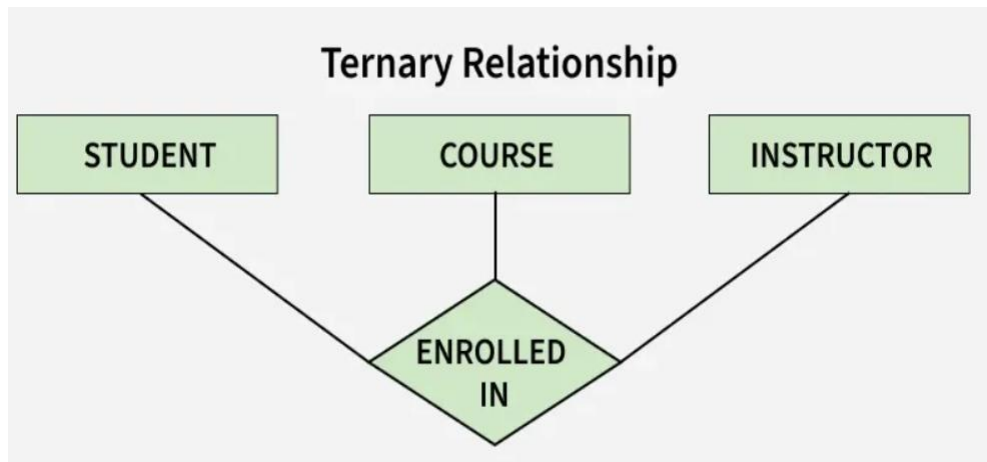
2. Binary Relationship:

- When there are TWO entities set participating in a relationship, the relationship is called a binary relationship.
- For example, a Student is enrolled in a Course.



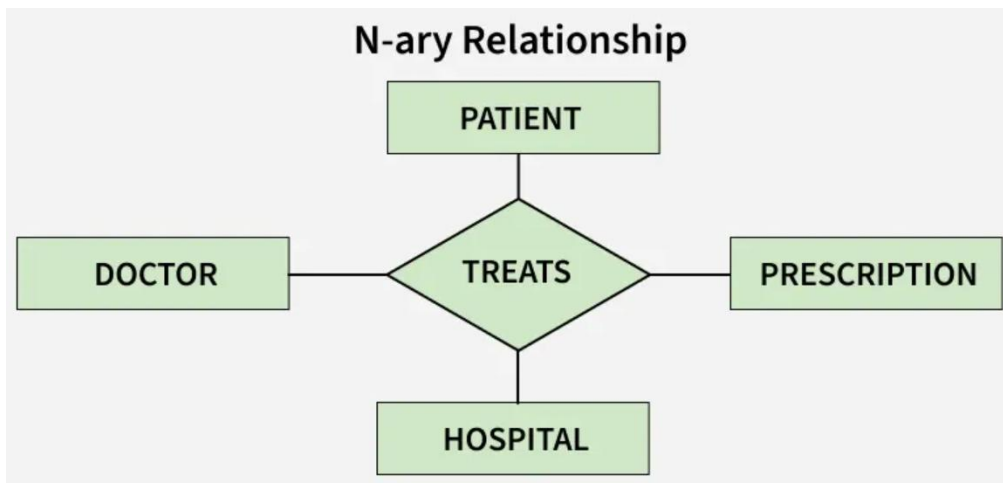
3. Ternary Relationship:

- When there are three entity sets participating in a relationship, the relationship is called a ternary relationship.



Ternary Relationship

4. N-ary Relationship: When there are n entities set participating in a relationship, the relationship is called an n-ary relationship.



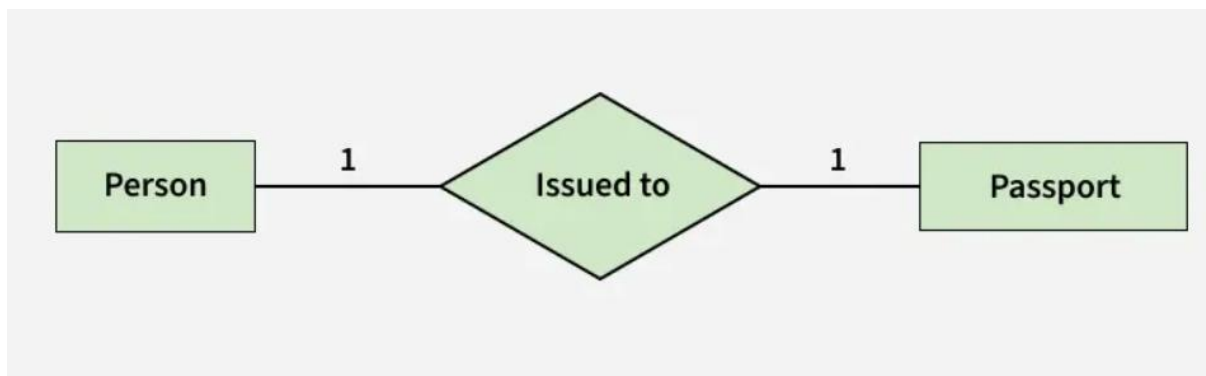
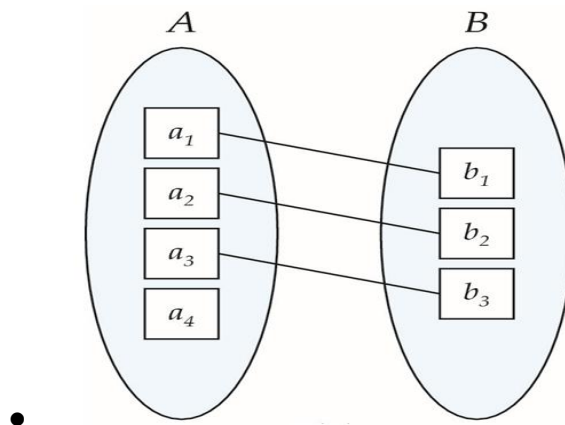
Cardinality in ER Model

The **maximum number of times an entity of an entity set participates in a relationship set** is known as cardinality.

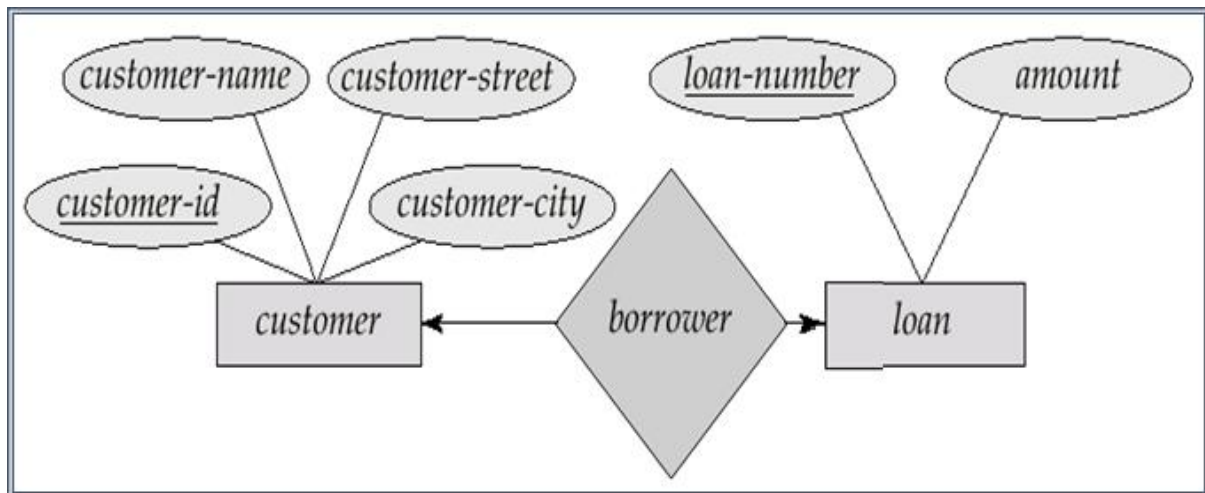
Cardinality can be of different types:

1. One-to-One

- When **each entity in each entity set can take part only once in the relationship**, the cardinality is one-to-one.
- Let us assume that **one person can be issued only one passport, and one passport is issued to only one person**.
- So, the relationship will be One-to-One (1 : 1), meaning that each person has a single passport, and each passport belongs to a single person.



one to one cardinality

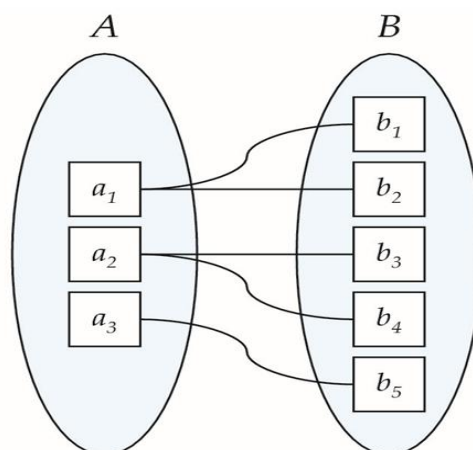


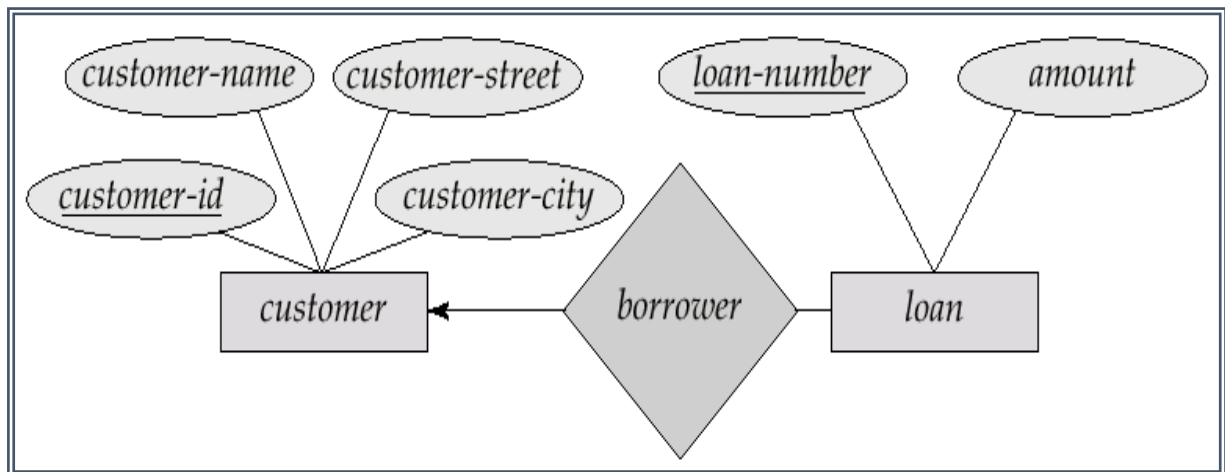
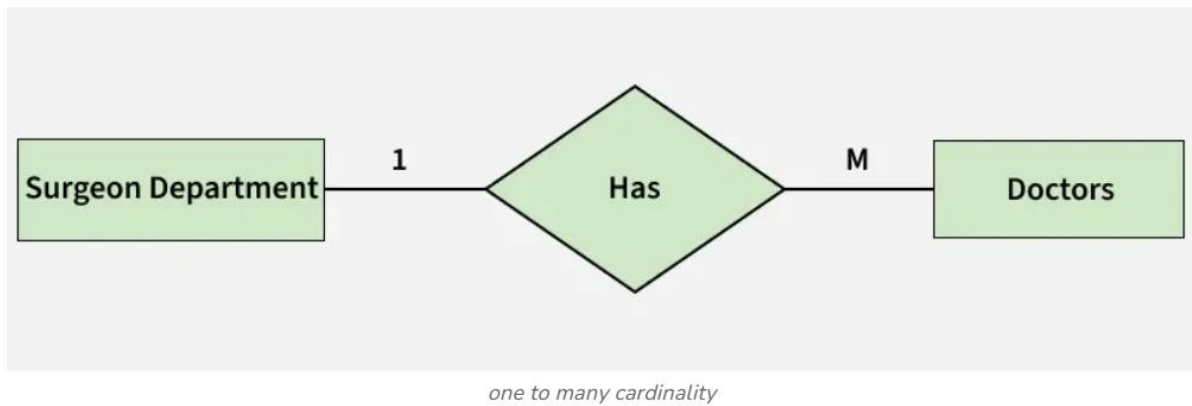
NOTE:

A directed line ($\rightarrow$), signify “one,” or an undirected line ($—$), signify “many”

2. One-to-Many

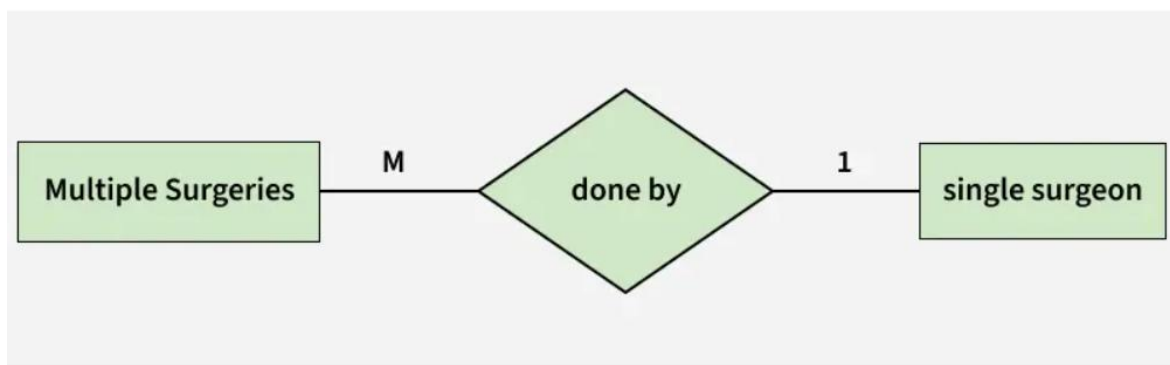
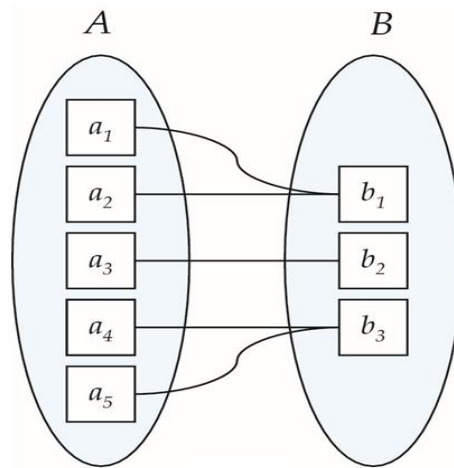
- In a one-to-many relationship, **one entity can be associated with multiple entities.**
- For example, a **single Surgeon Department can have many Doctors.**
- Therefore, the cardinality of this relationship is 1 to M, meaning one department can have many doctors.



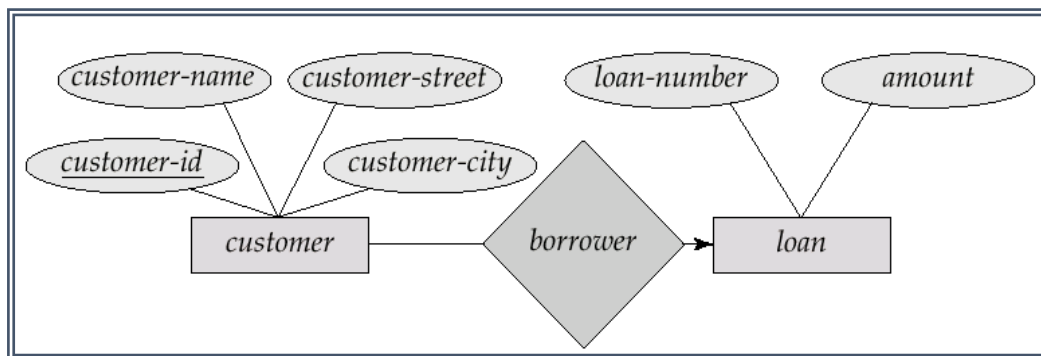


3. Many-to-One

- When **entities in one entity set can take part only once in the relationship set**, and **entities in other entity sets can take part more than once in the relationship set**, cardinality is many-to-one.
- Let us assume that **multiple surgeries can be performed by one surgeon, but one surgery is performed by only one surgeon.**
- So, the cardinality will be M to 1, meaning that many surgeries can be done by a single surgeon, but each surgery is done by only one surgeon.



many to one cardinality



Example 1: Employee – Department

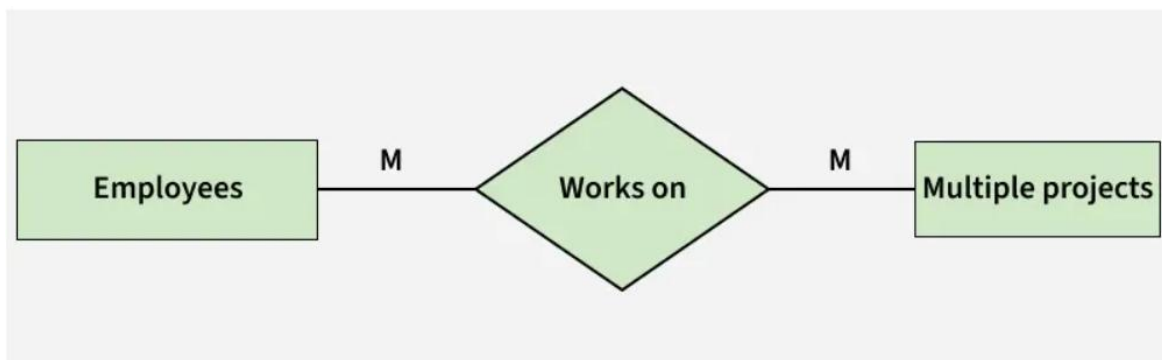
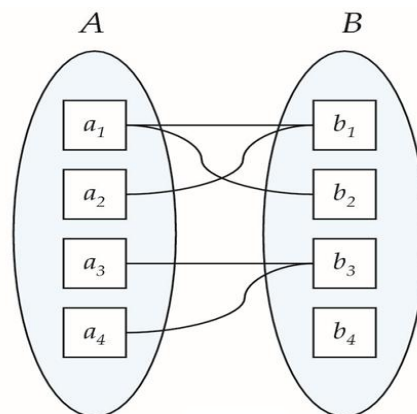
- Many employees work in one department
- Each employee belongs to only one department

Example 2: Student – College

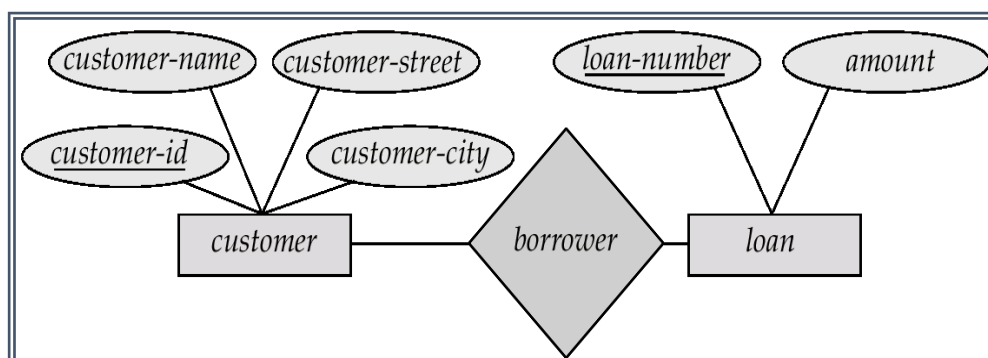
- Many students study in one college
- Each student is enrolled in only one college

4. Many-to-Many

- When **entities in all entity sets can take part more than once in the relationship** cardinality is many to many.
- Let us assume that an **employee can work on multiple projects and each project can have multiple employees working on it.**
- So, the relationship will be many-to-many (M:N), meaning that one employee may be associated with several projects, and one project may involve several employees.



many to many cardinality



Participation Constraint

Participation Constraint is applied to the entity participating in the relationship set.

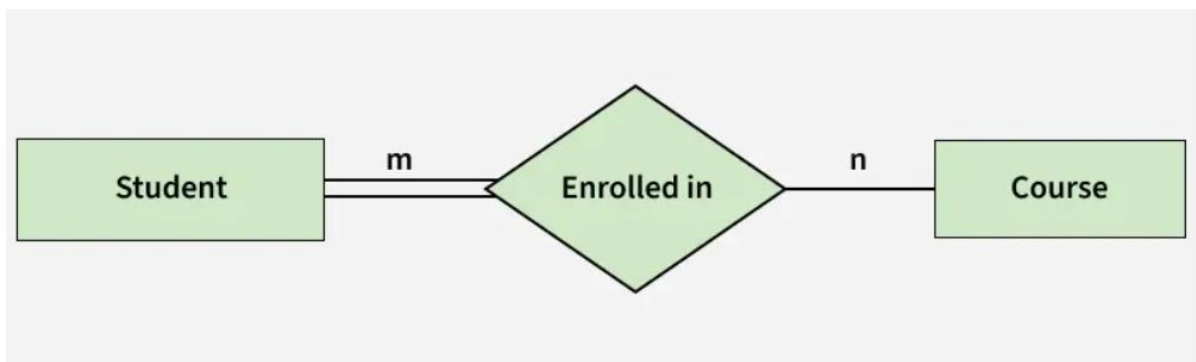
1. Total Participation:

- Each entity in the entity set must participate in the relationship.
- If *each student must enroll in a course, the participation of students will be total.*
- **Total participation is shown by a double line in the ER diagram.**

2. Partial Participation:

- The entity in the entity set may or may NOT participate in the relationship.
- If *some courses are not enrolled by any of the students*, the participation in the course will be partial.

The diagram depicts the 'Enrolled in' relationship set with Student Entity set having total participation and Course Entity set having partial participation.



Total Participation and Partial Participation

How to Draw an ER Diagram

1. **Identify Entities:** The very first step is to identify all the Entities. Represent these entities in a Rectangle and label them accordingly.
2. **Identify Relationships:** The next step is to identify the relationship between them and represent them accordingly using the Diamond shape. Ensure that relationships are not directly connected to each other.
3. **Add Attributes:** Attach attributes to the entities by using ovals. Each entity can have multiple attributes (such as name, age, etc.), which are connected to the respective entity.
4. **Define Primary Keys:** Assign primary keys to each entity. These are unique identifiers that help distinguish each instance of the entity. Represent them with underlined attributes.
5. **Remove Redundancies:** Review the diagram and eliminate unnecessary or repetitive entities and relationships.
6. **Review for Clarity:** Review the diagram make sure it is clear and effectively conveys the relationships between the entities.